

Contents

I	DataStructures	3
1	Lecture I	4
1.1	The Static Predecessor problem	4
2	Lecture II	15
2.1	Order Statistics problem	15
3	Lecture I/II/III	17
3.1	Basic Definitions	17
3.2	Imitation Learning	17
4	Lecture IV	21
4.1	Actor Critic methods	21
4.2	Offline RL outline	24
4.3	Estimated expected return	24
4.4	A full Algorithm walkthrough	27
II	DiscreteMathematics	28
5	Lecture I	29
5.1	Counting	29
6	Definitions	30
6.1	Distributions	30
6.2	Variational Auto Encoder	31
6.3	Evidence Lower bound	31

III	Stanford CME 296 - Diffusion/Large Vision Models	32
7	Lecture I	33
7.1	Introduction	33
8	Lecture 2	38
8.1	Introduction	38
8.2	Recap	38
8.3	Score Matching	39
8.4	Score Matching	41
8.5	Score of a Gaussian Distribution (example in 1D)	42
8.6	Denoising Score Matching	43
8.7	Limitations of vanilla Denoising Score Matching	43
8.8	Score Matching with varying noise via NCSN - Noise Conditional Sampling Network	43
8.9	This is Annealed Langevin Dynamics	44
8.10	Parallel between noise (DDPM) and score (NCSN)	45
8.11	Wiener Process	45
8.12	Training Derivation	46
9	Linear Dynamical Systems	48
9.1	Canonical Model for Time series Data	48
9.2	Main problem (informal)	49
9.3	Main problem (informal)	50
9.4	Prior work	50
9.5	observability and controllability	51
9.6	Blueprint	53

PART I

DataStructures

CHAPTER 1

Lecture I

1.1 The Static Predecessor problem

Data structure is represented by *Set* S of items $\{x_1, \dots, x_n\}$

Query : $\text{pred}(z) = \max\{x \in S : x \leftarrow z\}$

Example Solution : $\mathcal{O}(n \log n)$ using a balanced BST

Work RAM Model

- Item are integers $\{0, 1, \dots, 2^w - 1\}$
- “w” word size is $u = 2^w - 1$
- Pointers fit in a word
- Space $\geq n$ and $w \geq \log(\text{Space}) \geq \log(n)$

Two Data Structures

- Van Emde Boas Tree FOCS '75
 - Update/Query $\mathcal{O}(\log w)$ time
 - Update/Query $\mathcal{O}(\log u)$ space
- y Fast Tries Willard '83
- Fusion Trees Fredman, Willard JCCS '93
- Query is $\mathcal{O}(\log w^n)$ time
- Can achieve $\min(\lg w, \log w^n) \leq$

- Dynamic Fusion Trees $\mathcal{O}(n \lg n)$ sorting
- Dynamic Fusion Trees can also achieve $\mathcal{O}(n \lg \lg n)$ sorting (Han STOC '02) and $\mathcal{O}(n \sqrt{\lg \lg n})$ FOCS '02

Van Emde Boas

- Fields of $VEB_u(V)$
 - $\sqrt{u}$ size array $V.\text{cluster}[0] \dots V.\text{cluster}[\sqrt{u}-1]$
 - $V.\text{summary}$ is a $VEB_{\sqrt{u}}$
 - $V.\text{min}$ and $V.\text{max}$ are integers in $\{0, \dots, u-1\}$
- Representation of values
 - $X \in \{0, 1, \dots, u-1\}$
 -

$$x = \underbrace{1011}_c \overbrace{0011}^i$$

Predecessor Algorithm ($V, x = \langle c, i \rangle$)

```

if  $x > V.\text{max}$  :
    return  $V.\text{max}$ 
else if  $V.\text{cluster}[c].\text{min} < x$  :
    return  $\text{pred}(V.\text{cluster}[c], i)$ 
else  $c' = \text{pred}(V.\text{summary}, c)$  :
    return  $V.\text{cluster}[c'], \text{max}$ 

```

Insertion Algorithm ($V, x = \langle c, i \rangle$)

```
if  $V = \emptyset$  :  
     $V.\text{min} \leftarrow x^{\text{prime}};$   
if  $x < V.\text{min}$  :  
     $\text{swap}(x, V.\text{min});$   
else if  $V.\text{cluster}[c].\text{min} = \emptyset$  :  
     $\text{insert}(V.\text{summary}, c);$   
 $\text{insert}(V.\text{cluster}[c], i)$ 
```

Algorithm analysis ($V, x = \langle c, i \rangle$)

- Prediction time
 - $T(u) = T(\sqrt{u}) + \mathcal{O}(1)$
 - $T(u) = \mathcal{O}(\lg \lg u) = \mathcal{O}(\lg w)$
- Insertion time
 - $T(u) \leq 2T(\sqrt{u}) + \mathcal{O}(1)$
 - $T(w) \leq 2T\mathcal{O}\left(\frac{w}{2}\right) + \mathcal{O}(1)$

Space of VEB

$$S(u) = (\sqrt{u} + 1)S(\sqrt{u}) + \mathcal{O}(1) \rightarrow S(u) = \theta u$$

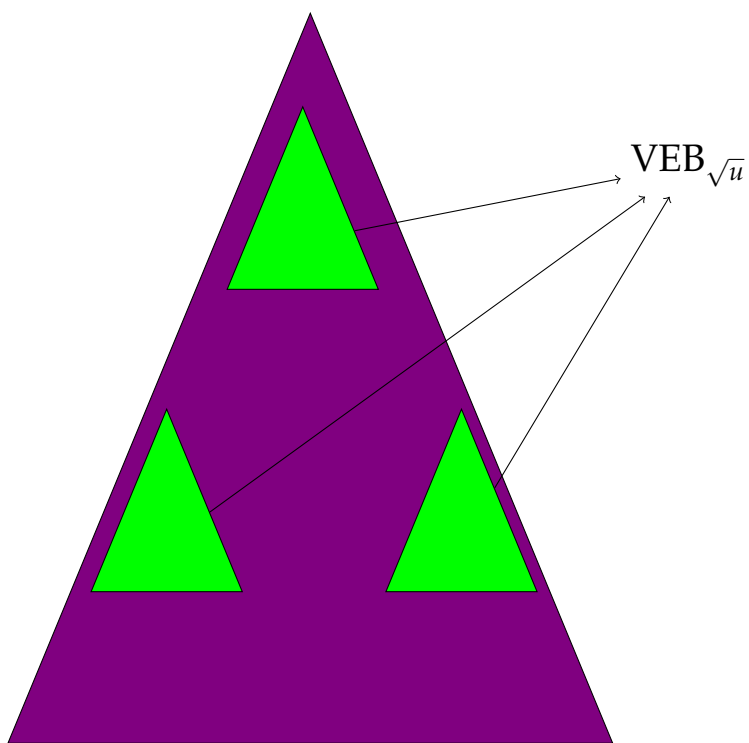


Figure 1.1: Van End Boas

Binary trie

Search time is $\mathcal{O}(\log n)$

- Red dashed lines are jump Pointers
- Jump pointers are used when children are missing to point to the successor

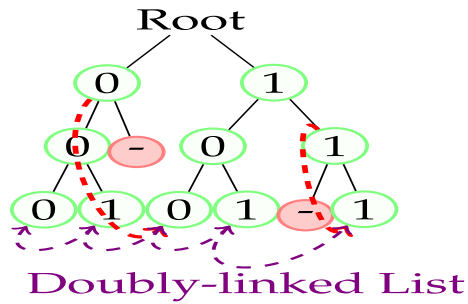


Figure 1.2: Binary Trie

Add node to Binary trie

Insertion time is $\mathcal{O}(\log n)$

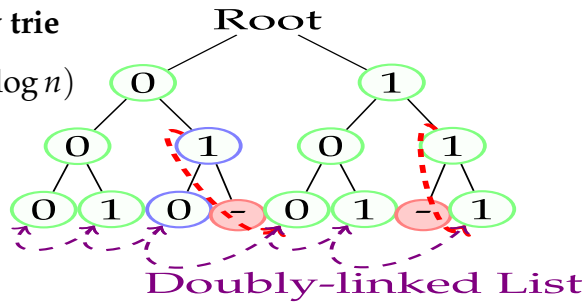


Figure 1.3: Binary Trie; Add Node

This shows an example of addition of a new node. The New node and its left child are in blue. The jump pointers are updated. And new node is added to the doubly-linked list at the appropriate position

The integer inserted above is

0 1 0

Remove node from Binary trie

Removal time is $\mathcal{O}(\log n)$

- Remove node (It will be a leaf)
- Walk back up and remove any node without a child and The jump pointers are updated. The doubly-linked list is updated.

x-fast trie

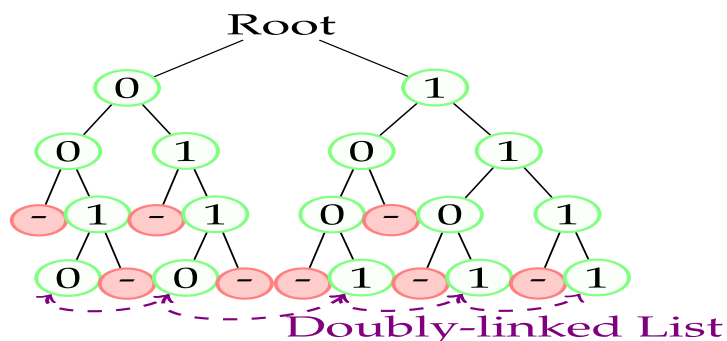


Figure 1.4: xfasttrie

- An x-Fast Trie is a threaded binary trie where leaves are stored in a doubly-linked list and where all nodes in each level are stored in a hash table.
- Can do lookups in the HashTable in time $\mathcal{O}(1)$.
- Claim: Can determine $\text{successor}(x)$ in time $\mathcal{O}(\log \log u)$.
- Start by binary searching for the longest prefix of x .
- If at a leaf node, follow the forward pointer to the successor.
- If at an internal node with a missing 1, follow the 1 thread.
- If at an internal node with a missing 0, follow the 0 thread and follow the forward pointer.

Fusion tries - Fredman, Willard JCSS '93

- Static Predecessor
- Made dynamic by Andersson, Thorup JACM ($\log_w n \log \log n$) deterministic updates

- Made dynamic by Raman ESA JACM '96 $\mathcal{O}(\log_w n)$ updates (Expected time since it is a randomized algorithm)
- Query for all $\mathcal{O}(\log_w n)$

Idea : B-Tree with branching factor $w^{1/5}$

- Fusion tree node stores $k = \mathcal{O}(w^{1/5})$ keys
- Items are integers $\{x_0, \dots, x_{k-1}\}$
- $\mathcal{O}(1)$ Predecessor/Successor
- Polynomial time $k^{0(1)}$ Preprocessing

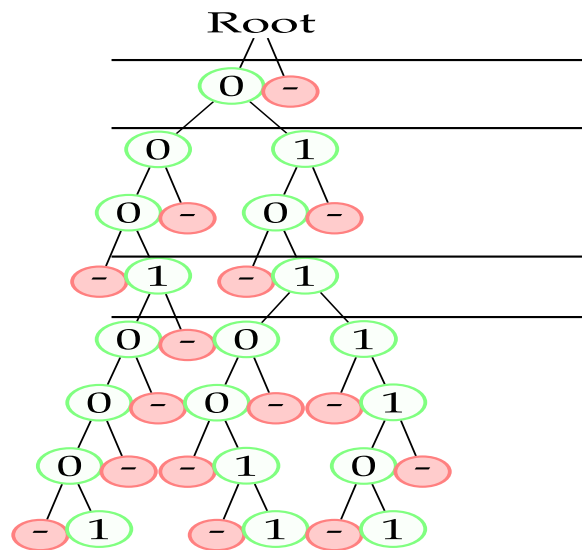


Figure 1.5: Fusion Trees Distinguishing Keys

The tree shown has

$$k-1$$

branching nodes and the height of the tree is

$$w$$

$\rightarrow \leq k-1$ levels containing a branching node. The levels starting from the top of the tree correspond to important bits the indices of which are $b_0, \dots, b_r$ where $r < k = \mathcal{O}(w^{1/5})$ The leaves have the least significant bits.

The parallel lines show that we can check some branching nodes to find out if the lower bits can exist or not instead of traversing upto the leaf. Refer the section on Sketching.

What is sketching in this algorithm?

$\text{sketch}(x) = \text{Extract bits } \{b_0, \dots, b_{r-1}\}$

$$\begin{array}{ccccccc} & & & & b_1 & & b_0 \\ & & & & \uparrow & & \uparrow \\ 1 & \underbrace{1} & 1 & \underbrace{0} & \underbrace{0} & 01 & \underbrace{1} \\ & b_3 & & b_2 & & & \end{array}$$

Sketching Example

- The important bits (violet circles) are used for sketching
- The labels of the leaf nodes also show these (violet) bits

The query is shown by the blue path.

$$\begin{array}{c} 0101 \\ \underbrace{\hspace{1.5cm}} \\ \text{Query} \end{array}$$

The sketch is

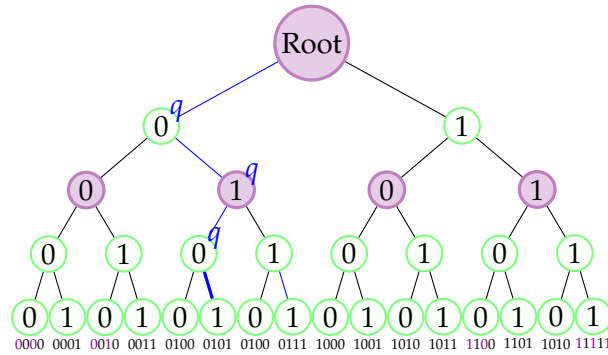
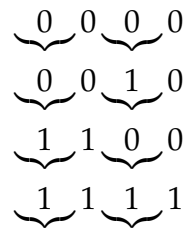


Figure 1.6: Example of Sketching



Since the sketch of the query **0101** matches the sketch **00** the answer will be the left-most node in the tree. This is wrong. It is not the predecessor **0010** or the successor **1100**

We need to find a solution for this.

Let us assume $\text{sketch}(x) \leq \text{sketch}(g) \leq \text{sketch}(x_{k+1})$. We will look for the longest common prefix/ancestor of the query

q

and either x_i or x_{i+1} .

Now we can identify the wrong path we took. The correct path from the LCA(Least common Ancestor) would have lead to the left, to the tree whose nodes are shown in grey. Identify the maximum in that tree. This is **e**. And find $\text{sketch}(e)$ among $\text{sketch}(x_i)$. Now we can find the predecessor $\text{sketch}(e)$ among $\text{sketch}(x_i)$. Similarly we find the successor using the same algorithm.

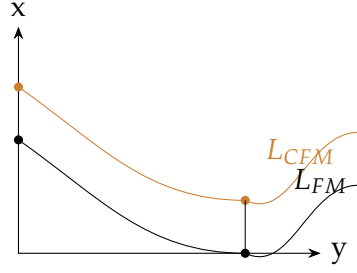


Figure 1.8: xfasttrie

Pick $m_i = m'_i + (w - b_i + i_{r^3})$ rounded down to r^3

We also need to mask with $(\sum_i^{r-1} 2^{b_i+m_i})$ and shift by $(b_0 + m_0)$

CHAPTER 2

Lecture II

2.1 Order Statistics problem

Given k elements in an array, find k th smallest element (element of k rank)

Sorting will take $\mathcal{O}(n \log n)$ and we want to better than that, ideally better than linear time.

We know that that median is $\left\lceil \frac{k+1}{2} \right\rceil$ or $\left\lfloor \frac{k+1}{2} \right\rfloor$

It is tricky to find the median in linear time. In addition we also find k th smallest element.

That is the focus of this lecture.

Randomized Divide and Conquer

Rand-select ($A - p - q - \underbrace{\quad}_i$)
 i th smallest element in A to

```

if  $p = q$  then return  $A[p]$  :
 $r \leftarrow$  Rand-partition  $A - p - q$ 
 $k \leftarrow r - p - 1$ 
else if  $V.\text{cluster}[c].\text{min} < x$  :
    return  $\text{pred}(V.\text{cluster}[c], i)$ 
else  $c' = \text{pred}(V.\text{summary}, c)$  :
    return  $V.\text{cluster}[c'], \text{max}$ 

```

Rand-partition $A - p - q$ means that we pick a value in the range $p-q$ and use it to partition the array, so that all elements less than the *random* element are on the left and the greater elements are on the right

CHAPTER 3

Lecture I/II/III

3.1 Basic Definitions

- state s_t , is the state of the world at time t
- action a_t , the decision taken at time t
- trajectory τ , sequence of states/observations and actions $D := (s_1, a_1, \dots, s_T)$
- reward function $r(s, a)$
- Policy $\pi(a, s)$ or $\pi(a, o)$

$$\min_{\theta} \mathbb{E}_{\tau \sim P_{\cdot}(\emptyset)} \left[\sum_t^T r(s_t, a_t) \right]$$

3.2 Imitation Learning

What is the goal of imitation learning ?

- Data: Given trajectories collected by an expert “demonstrations” $D := (s_1, a_1, \dots, s_T)$
- Sampled from some unknown policy π_{expert}
- Goal: Learn a policy π_{θ} that performs at the level of π_{expert} by mimicking

Imitation learning -Version 0

Deterministic policy

- Data: Given trajectories collected by an expert “demonstrations” $D := (s_1, a_1, \dots, s_T)$

- Supervised regression to the expert's policy $\min_{\theta} \frac{1}{\|\mathcal{D}\|} \sum_{(s,a) \in \mathcal{D}} \|a - \hat{a}\|_2^2$
- Deploy learned policy π_{θ}

Evaluation of the RL objective

$$\theta^* = \operatorname{argmax}_{\theta} \underbrace{\mathbb{E}_{\tau \sim P_{\theta}} \left[\sum_t^T r(s_t, a_t) \right]}_{J(\theta)}$$

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int P_{\theta}(\tau) r(\tau) d\tau = \int \nabla_{\theta} P_{\theta}(\tau) r(\tau) d\tau$$

This gradient can be simplified using this trick.

$$= P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau)$$

$$= P_{\theta}(\tau) \frac{\nabla_{\theta} P_{\theta}(\tau)}{P_{\theta}(\tau)}$$

We apply the chain rule after evaluating the derivation of $\log P_{\theta}(\tau)$ which is $\frac{1}{P_{\theta}(\tau)}$

This means that $\int \nabla_{\theta} P_{\theta}(\tau) r(\tau) d\tau$ is $= \int \nabla_{\theta} \log P_{\theta}(\tau) r(\tau) d\tau$

We know that $P_{\theta}(\tau) = P(s_t) \prod_t \pi_{\theta}(a_t, s_t) P(s_{t+1} \mid (s_t, a_t))$

So $\log P_{\theta}(\tau) = \log P(s_t) + \sum_t \log \pi_{\theta}(a_t, s_t) + \log P(s_{t+1} \mid (s_t, a_t))$

$\log P(s_t)$ and $\log P(s_{t+1} \mid (s_t, a_t))$ are constants.

$$\nabla \log P_{\theta}(\tau) = \nabla_{\theta} \sum_t \log \pi_{\theta}(a_t, s_t)$$

Expectation over trajectories sampled by our policy is
 $\mathbb{E}_{\tau \sim P_{\theta}(\tau)}$

So the policy gradient now is $\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} \left[\sum_t^T \nabla_{\theta} \log \pi(a_t, s_t) \right] \left[\sum_t^T r(s_t, a_t) \right]$

The equation shown above is the gradient of the policy of the RL objective.

Estimating the Gradient

This is the full algorithm.

1. Sample r_i from $\pi_{\theta}(a_t | s_t)$
2. $\nabla_{\theta} J(\theta)$ from $\pi_{\theta}(a_t | s_t) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i) \right) \left(\sum_t (a_t^i s_t^i) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$
1. The equation in the first box is for imitation gradient, weighted by reward
2. The 2nd equation is for imitation learning

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i^N \left(\sum_i^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} s_{i,t}) \right) \left(\sum_i^T r(s_{i,t}, a_{i,t}) \right)$$

$$\nabla_{\theta} J_{BC}(\theta) \approx \frac{1}{N} \sum_i^N \left(\sum_i^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} s_{i,t}) \right)$$

Figure 3.1: Imitation gradient and learning

This increases the likelihood of action when the rewards are high and decreases the likelihood of actions in trajectories with low reward.

Off-policy version of Policy Gradient

Importance Sampling - Using proposal distribution q , we get $\mathbb{E}_{x \sim P(x)}[f(x)] = \int P(x)f(x)dx = \int \frac{q(x)}{q(x)}P(x)f(x)dx = \int q(x)\frac{p(x)}{q(x)}f(x)dx = \mathbb{E}_{x \sim q(x)}\left[\frac{p(x)}{q(x)}\right]f(x)$
 Please note: Important for q to have non-zero support for high probability $P(X)$

Figure 3.2: Off-policy Gradient

$q(x)$ is considered the old policy. $\mathbb{E}_{x \sim q(x)}\left[\frac{p(x)}{q(x)}\right]f(x)$ and we are sampling from the old policy and weighting by dividing the new policy by the old policy.

What if we want to use samples from $\bar{P}(\tau)$?

$$J(\theta) = \mathbb{E}_{\tau \sim \bar{P}_\theta(\tau)}\left[\frac{P_\theta(\tau)}{\bar{P}(\tau)}\right]r(\tau)$$

Some of the probabilities cancel out.

$$\frac{P_\theta(\tau)}{\bar{P}(\tau)} = \frac{\cancel{P(s_1)}^{\Pi_{t=1}^T \pi_\theta(a_t, s_t)} P(s_{t+1} | (s_t, a_t))}{\cancel{P(s_1)}^{\Pi_{t=1}^T \bar{\pi}_\theta(a_t, s_t)} P(s_{t+1} | (s_t, a_t))}$$

Common final form $\nabla_{\theta'} J(\theta') \approx$

$$\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{i \pi_\theta(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right) - b \right)$$

Figure 3.3: Off-policy Gradient

CHAPTER 4

Lecture IV

4.1 Actor Critic methods

Value function

This is the future rewards starting at s and following policy π

$$V^\pi(s_t) = \sum_{t'=t}^T \mathbb{E}_{s_{t'}, a_{t'} \sim \pi} [r(s_{t'}, a_{t'}) \mid s_t]$$

Q-function Q^π

$$Q^\pi(s_t, a_t) = \sum_{t'=t}^T \mathbb{E}_\pi [r(s_{t'}, a_{t'}) \mid s_t, a_t]$$

Improving Policy Gradients

The gradient would be better if we increased the likelihood when Q is high and decreased the likelihood when Q is low

$$\nabla_\theta J(\theta) \sim \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_{i,t} | s_{i,t}) Q(s_{i,t}, a_{i,t})$$

The explanation shown below is about this.

Introducing baselines

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} \left[\nabla_{\theta} \log P_{\theta}(\tau) r(\tau) - \underbrace{b}_{\substack{\text{subtracting} \\ \text{baseline} \\ \text{constant}}} \underbrace{a}_{\text{“con-stant”}} \right]$$

which is the average reward given by

$$b = \frac{1}{N} \sum_{i=1}^N r(\tau)$$

If we subtract average reward we get negative-gradients for below-average behaviour.

The following is the proof that the subtracted value doesn't have any effect on the gradient

$$\mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(\tau) b] = \int p_{\theta}(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau) b d\tau \text{ Using this trick}$$

$$P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau) = \nabla_{\theta} P_{\theta}(\tau)$$

$$\text{we get} = b \underbrace{\nabla_{\theta} \int P_{\theta}(\tau) d\tau}_{\substack{\text{This is 1} \\ \text{Gradient is zero}}}$$

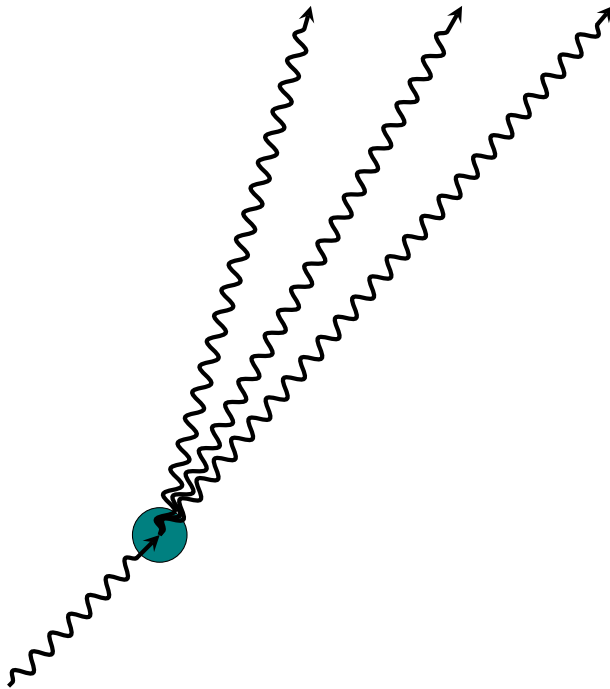


Figure 4.1: Improving policy Gradients

This will ameliorate the situation caused when the robot takes a small step forward and fall backwards.

Estimating future rewards using Q function

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t})$$

$$r(s_{i,t'}, a_{i,t'})$$

Estimate future
rewards if we
take a_t at s_t

True future rewards can be obtained by using the Q function like this.

We will get much better results if the form of the gradient is like this.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}) Q(a_{i,t}, s_{i,t})$$

What about baselines ?

Once we start using the Q function we get this when we subtract the baseline.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}) (\mathcal{Q}(a_{i,t}, s_{i,t}) - \underbrace{b}_{\mathbb{V}^{\pi}(s_t)})$$

$$b = \text{average reward} = \frac{1}{N} \sum_i \mathcal{Q}(s_{i,t}, a_{i,t})$$

$$V(s_t) = \mathbb{E}_{a_t \sim \pi_{\theta}(\cdot \mid s_t)} [\mathcal{Q}(s_t, a_t)]$$

$$\text{Recall : } \mathbb{A}^{\pi}(a_t, s_t) = \mathcal{Q}^{\pi}(a_t, s_t) - \mathbb{V}^{\pi}(s_t)$$

So with baseline we get

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}) \mathbb{A}^{\pi}(a_t, s_t)$$

4.2 Offline RL outline

Better estimates of $\mathbb{A}$ will lead to less noisy gradients.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}) \mathbb{A}^{\pi}(a_t, s_t)$$

First initialize the policy (randomly with imitation learning and heuristics)

4.3 Estimated expected return

Should we fit $\mathbb{V}^{\pi}, \mathcal{Q}^{\pi}, \mathbb{A}^{\pi}$? We are going to fit $\mathbb{V}^{\pi}$.

Recall : $\mathbb{A}^{\pi}(s_t, a_t) = \mathcal{Q}^{\pi}(s_t, a_t) - \mathbb{V}^{\pi}(s_t)$. There is a way to turn this equation that only depends on $\mathbb{V}^{\pi}$. $\mathcal{Q}^{\pi}(a_t, s_t)$ starts at state s , takes action a and sums the rewards. So we can write the equation like this.

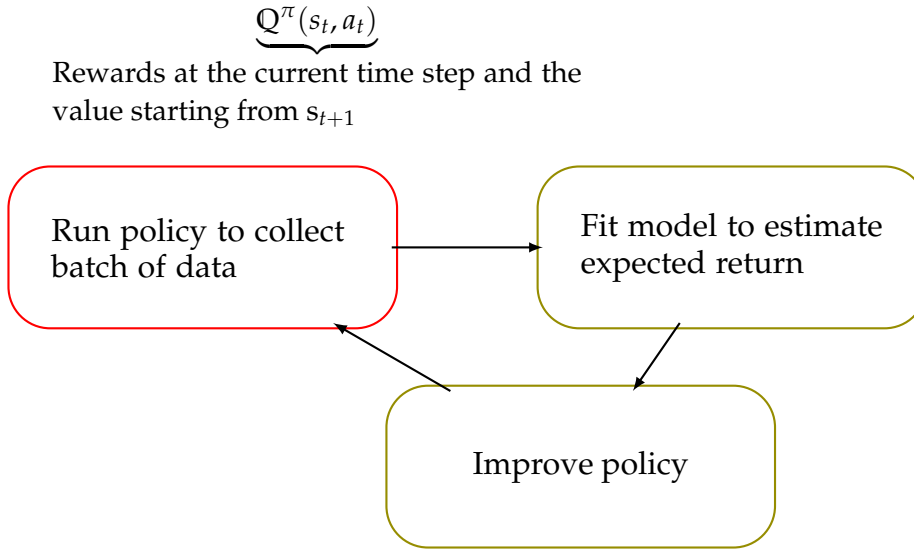


Figure 4.2: Online RL

So we can rewrite the equations like this.

$$Q^\pi(s_t, a_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_\theta} [r(s_{t'}, a_{t'}) \mid (s_t, a_t)]$$

$$r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim P(\bullet \mid (s_t, a_t))} [\mathbb{V}_{s_{t+1}}^\pi]$$

Use the sampled s_{t+1}

$$\mathbb{A}^\pi(s_t, a_t) = r(s_t, a_t) + \mathbb{V}^\pi(s_{t+1}) - \mathbb{V}^\pi(s_t)$$

Estimating $\mathbb{V}^\pi$

Version 1 : Monte carlo estimation

1. Original single sample estimate $\mathbb{V}^\pi(s_t) = \sum_{t'=t}^T r(s_{t'}, a_{t'})$
2. Multisample estimate (but can't reset the world in practice) $\mathbb{V}^\pi(s_t) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \sum_{t'=t}^T r(s_{t'}, a_{t'})$

Step 1 : Aggregate dataset of single sample estimates

$$\left(s_{i,t}, \underbrace{\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'})}_{y_{i,t}} \right)$$

Step 2: $\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(s_i) - y_i \right\|_2^2$ Laplace transform

Version 2 : Bootstrapping

Ideal target is the true sum of rewards. This is equal to the reward at the current time step and the expected reward at the next time step.

$$\text{Monte carlo target : } y_{i,t} = \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'})$$

$$\begin{aligned} \text{Ideal target : } & \sum_{t'=t}^T \mathbb{E}_{\pi_\theta} [r(s_{i,t'}, a_{i,t'}) \mid (s_{i,t})] \approx \\ & r(s_{i,t}, a_{i,t}) + \sum_{t'=t+1}^T \mathbb{E}_{\pi_\theta} [r(s_{i,t'}, a_{i,t'}) \mid (s_{i,t+1})] \approx \\ & r(s_{i,t}, a_{i,t}) + \underbrace{\mathbb{V}^\pi(s_{i,t+1})}_{\hat{V}_\phi^\pi(s_{i,t+1})} \approx r(s_{i,t}, a_{i,t}) + \underbrace{\hat{V}_\phi^\pi(s_{i,t+1})}_{\text{Fitted value function}} \end{aligned}$$

Step 1 : Aggregate dataset of “bootstrapped” estimates This means that we are bootstrapping off of our own estimate of the value function as part of the learning process.

$$\text{training data : } \left(s_{i,t}, r(s_{i,t}, a_{i,t}) + \underbrace{\hat{V}_\phi^\pi(s_{i,t+1})}_{\text{Fitted value function}} \right) \text{ Up-}$$

date labels every gradient step

Step 2 : Supervised learning to fit estimated value function. This is also known as temporal difference learning. $\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(s_i) - y_i \right\|_2^2$

4.4 A full Algorithm walkthrough

1. Sample batch of data $(s_1, a_1, \dots, s_T, a_T)$ from π_θ
2. $\mathbb{V}_\phi^{\pi_\theta}$ to summed rewards of data
3. $\hat{A}^{\pi_\theta}(s_{t,i}, a_{t,i}) + \hat{\mathbb{V}}_\phi^{\pi_\theta}(s_{t+1,i}) + \hat{\mathbb{V}}_\phi^{\pi_\theta}(s_{t,i})$
4. $\nabla_\theta J(\theta) \approx \sum_{t,i} \nabla_\theta \log \pi_\theta(a_{t,i} | s_{t,i}) \hat{A}^{\pi_\theta}(s_{t,i}, a_{t,i})$

$$y_{i,t+1} \sum_{t'=t}^{t+n+1} r(s_{i,t'}, a_{i,t'}) + \gamma^n \hat{\mathbb{V}}_\phi^{\pi_\theta} s_{i,t+n}$$

$$\frac{1}{2} \sum_i \left\| \hat{\mathbb{V}}_\phi^\pi(s_i) - y_i \right\|_2^2 \forall t, i$$

PART II

DiscreteMathemati

CHAPTER 5

Lecture I

5.1 Counting

Binomial Co-efficients

$\binom{n}{k}$ - No: of ways to choose 'n' things from 'k' objects.

CHAPTER 6

Definitions

6.1 Distributions

Marginal distribution

The marginal distribution of a subset of a set of random variables is the probability distribution of the variables contained in the subset

$$p(x) = \int p(x, \underbrace{\quad}_{z}) dz$$

Taking the integral over z to marginalize x

Example for $p(x) = 1$

		y		
		1	2	3
x	1	0.32	0.03	0.01
	2	0.06	0.24	0.02
	3	0.02	0.03	0.27

Expected value

The expected value is the arithmetic mean of the possible values a random variable can take, weighted by the probability of those outcomes.

$$\mathbb{E}_{p(x)} = \int x p(x) dx$$

Bayes Theorem

$$p(x \div z) = \text{left} \left(\frac{p(z \div x) p(x)}{p(z)} \right) p(x \div z) p(x) = p(z \div x) p(x) = \underbrace{p(x, z)}_{\text{Joint probability}} = \frac{0.03}{0.36}$$

This is equal to the Joint probability.

		y		
		1	2	3
x	1	0.32	0.03	0.01
	2	0.06	0.24	0.02
	3	0.02	0.03	0.27

Jensen's inequality

f is a convex function if $f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \forall x_1, x_2$ and $t \in [0, 1]$

3D points

$$f(\sum_i t_i x_i) \leq \sum_i t_i f(x_i)$$

Replacing data points with probabilities

If we have probabilities then it is the same as before and the equation is this.

$$f(\mathbb{E}_{p(x)}[x]) \leq \mathbb{E}_{p(x)}[f(x)]$$

6.2 Variational Auto Encoder

For all given real image x , we want to maximize the marginal probability

$$p(x) = \int p(x, z) dz = \int p(x | z) p(z) dz$$

How to compute the integral? It is intractable.

$$p(x) = \frac{p(x, z)}{p(z|x)} = \frac{p(x|z)p(z)}{p(z|x)}$$

6.3 Evidence Lower bound

PART III

Stanford CME
296 - Diffu-
sion/Large
Vision Models

CHAPTER 7

Lecture I

7.1 Introduction

Meaning of Probability in Higher Dimensions

$x \sim \mathcal{N}(\sigma, \Sigma)$ All of these values are vectors and matrices in high dimensions.

$$x^{(i)} \sim \mathcal{N}(\sigma^{(i)}, \Sigma^{(i,i)}) \text{ and } \text{Cov}(x^{(i)}, x^{(i)}) = \Sigma^{(i,i)}$$

But we will be dealing only with **Isotropic** Gaussian matrices.

$$\Sigma = \sigma^2 \mathcal{I} = \begin{bmatrix} \sigma^2 & 0 & 0 & \dots & 0 \\ 0 & \sigma^2 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma^2 \end{bmatrix}$$

This is to simplify the math so that the **variance** is same in all directions.

Mathematical formalism of the forward Process

$$q(x^t | x^{t-1}) = \mathcal{N}(\sqrt{(1 - \beta_t)}(x_{t-1}), \beta_t \mathcal{I})$$

The following needs more explanation which may appear later in the document.

The following is just parameterization.

$$x_t = \underbrace{(\sqrt{(1 - \beta_t)}(x_{t-1}))}_{\alpha_t} + \underbrace{\sqrt{\beta_t} \epsilon_t}_{\alpha_t}$$

Multiplying we get

$$(\sqrt{\alpha_t \alpha_{t-1}})x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-1} + \sqrt{(1 - \alpha_t)}\epsilon_t$$

It appears that two random variables drawn from the same Gaussian are combined to get a variance like this.

The two quantities are **squared** and added.

$$\alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = \underbrace{1 - \alpha_t \alpha_{t-1}}_{\text{Variance}}$$

Since we combine random variables like this

we get

Stanard Deviation of the Variance shown above

$$(\sqrt{\alpha_t \alpha_{t-1}})x_{t-2} + \underbrace{(\sqrt{1 - \alpha_t \alpha_{t-1}})}_{\text{Variance}} \epsilon$$

Therefore we can have a noisy image at step **t** as a function of the clean image at step **0**

$$q(x^t | x^{t-1}) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, 1 - \bar{\alpha}_t\mathbf{I})$$

with

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i \text{ and } \alpha_t = 1 - \beta_t$$

Variance of sum of independent Gaussians is sum of respective variances

Objective

$$\underbrace{\max_{\theta}}_{\substack{\uparrow \\ \text{Find } \theta \text{ that} \\ \text{maximizes}}} \underbrace{\log P_{\theta}}_{\substack{\uparrow \\ \text{log likelihood} \\ \text{under model}}} \underbrace{(x_0)}_{\substack{\uparrow \\ \text{training} \\ \text{data}}}$$

Joint Probability Distribution

$$P(x_1, x_2) = P(x_1) \underbrace{P(x_1 \mid x_2)}_{\text{Conditional Probability}}$$

Marginalization

$$P(x_1) = \int P(x_1 \mid x_2)$$

It appears that we couldn't possibly marginalize over all possible trajectories using $\log P_\theta(x_0) = \log \int P_\theta(x_0, x_{1:T}) dx_{1:T}$

This is intractable.

Introduction of Evidence Lower Bound

$$= \int \left(\frac{P_\theta(x_{0:T}) \underbrace{q(x_{1:T}, x_0)}_{\text{Forward process we have defined given the clean image } x_0}}{q(x_{1:T}, x_0)} \right) dx_{1:T}$$

This is the same as the expectation which is written like this.

$$= \mathbb{E}_{\underbrace{x_{1:T} \sim q(x_{1:T}, x_0)}_{\text{Sampled}}} \left(\frac{P_\theta(x_{0:T})}{q(x_{1:T}, x_0)} \right)$$

Jensen's Inequality

$$= \log \left[\mathbb{E} \left(\frac{P_\theta(x_{0:T})}{q(x_{1:T}, x_0)} \right) \right] \geq \mathbb{E} \left[\log \left(\frac{P_\theta(x_{0:T})}{q(x_{1:T}, x_0)} \right) \right]$$

It appears that due to the chain rule the actual equation is like this. This has to be learnt again.

$$= \mathbb{E}_{x_0 \sim q(x_0)} (\log P_\theta(x_0)) \geq \mathbb{E}_{x_{0:T} \sim q(x_{0:T})} \left[\log \left(\frac{P_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right) \right]$$

This also seems to indicate that we don't sum over all the trajectories but only those sampled from the forward process q .

KL Divergence

$$\text{ELBO} = -\sum_{t=2}^T \text{KL}(q(x_{t-1} | x_t, x_0) \parallel P_\theta(x_{t-1} | x_t)) + \text{Some other terms}$$

Show Lower Bound is tractable

$$q(\overbrace{x_{t-1}}^{\text{Less Noisy}} \mid \underbrace{x_t}_{\text{More noisy image}}, \underbrace{x_0}_{\text{Clean image}}) \text{ is tractable}$$

The question is this. What is the distribution of the less noisy image given the more noisy image along with the clean image ?

Bayes' Rule

$$\underbrace{P(A \mid B)}_{\text{Posterior}} = \frac{P(B|A)P(B)}{\underbrace{P(B)}_{\text{Marginal Likelihood}}}$$

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t|x_{t-1},x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

We can write these equations now based on Bayes' rule.

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t|x_{t-1},x_0) \overbrace{q(x_{t-1} \mid x_0)}^{\text{Tractable}}}{\underbrace{q(x_t \mid x_0)}_{\text{Tractable and fixed (Constant)}}}$$

q is markovian

$$\text{And } q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1})q(x_t \mid x_0)$$

As far as the right side of the approximation shown above is concerned, the product of the probabilities of two Gaussians is also Gaussian.

So the first term in the **ELBO** derivation is tractable. Now we have to show that the second term is too.

We assume P_θ is Gaussian. $P_\theta(x_{t-1} \mid x_t) = \mathbf{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$

Justification If $q(x_t \mid x_{t-1})$ is Gaussian, then $q(x_{t-1} \mid x_t)$ is approx. Gaussian

Final Loss

$$\mathcal{L}_{DDPM}(\theta) = \mathbb{E}_{t \sim Unif, x_0 \sim q_0(x_0), \epsilon \sim \mathcal{N}(0, I_d)} [\|\epsilon^\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t - \epsilon)\|_2^2]$$

This seems to mean that if we have a noisy images and the original noise, we can compute the actual noise that was added.

CHAPTER 8

Lecture 2

8.1 Introduction

8.2 Recap

$$\mathcal{L}_{DDPM}(\theta) = \mathbb{E}_{t \sim \text{Unif}, x_0 \sim q_0(x_0), \epsilon \sim \mathcal{N}(0, I_d)} \left[\left\| \underbrace{\text{Noise predictor}}_{\epsilon^\theta} \left(\underbrace{\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon}_{\text{Noised image}}, \underbrace{t}_{\text{Noise level}} - \underbrace{\epsilon}_{\text{Actual level}} \right) \right\|_2^2 \right]$$

8.3 Score Matching

Intuition

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

The space has multiple dimensions with larger values of $P(x)$ in some areas and smaller values in others.

The question is this. How do we move x to a point where $P(x)$ is higher.

We need to calculate the gradient.

The gradient of a scalar function that takes as input, x , which is multi-dimensional and returns a scalar is a vector of partial derivatives of the scalar in each dimension.

The gradient represents the direction where value of $P(x)$ is higher.

If we access the gradient of $P(x)$ we can move from x to a region where $P(x)$'s value higher.

Problem with gradient of probabilities

Probabilities are normalized. They sum to one

Property of probabilities $\mathcal{Z} = \int \mathcal{P}_{data} x dx = 1$

Therefore

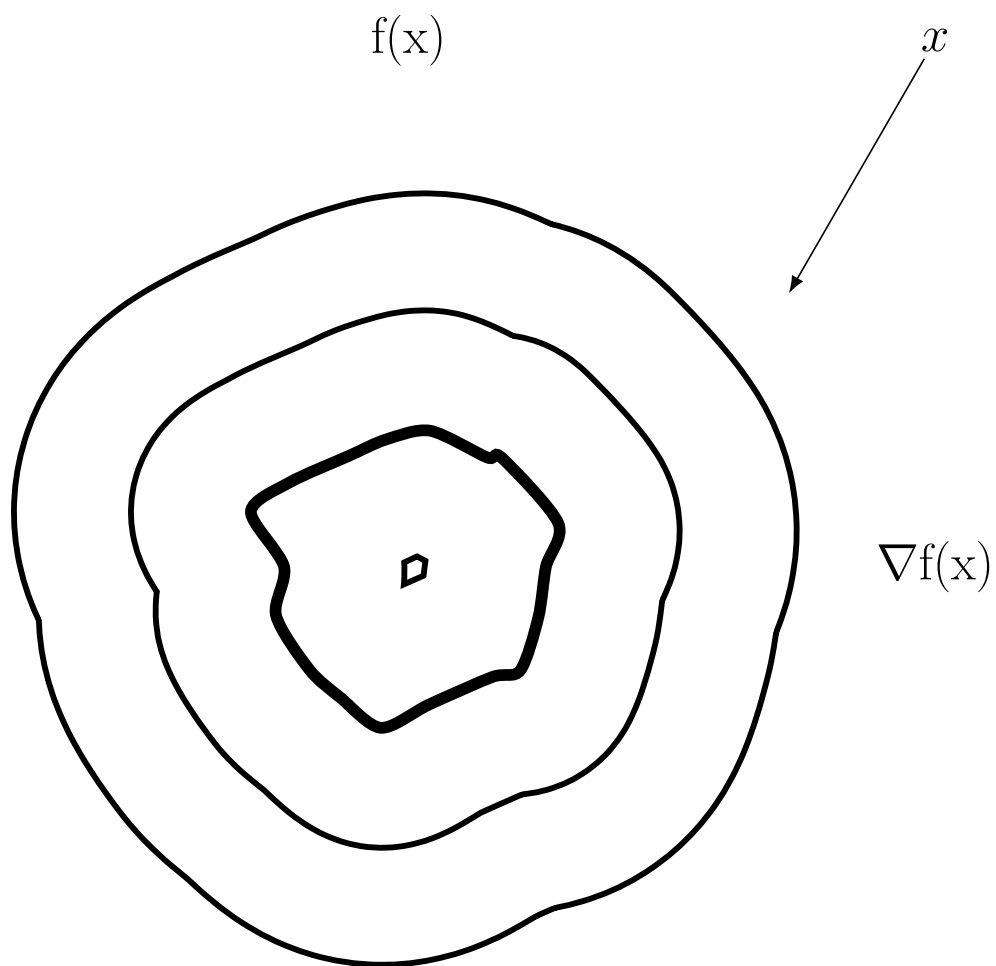


Figure 8.1: Gradient of Probability Distribution function

$$\mathcal{P}_{data} = \left(\frac{f(x)}{\underbrace{Z}_{\text{intractable}}} \right)$$

It is intractable because we cannot sum across multiple dimensions. Moreover the PDF can encounter numerical stability issues in some regions where the probabilities are very small.

Gradient of log of probabilities

1. $\nabla_x \log \mathcal{P}_{data}(x) = \nabla_x \log f(x) \nabla_x \log(Z)$
2. Points in the same direction as gradient of $\mathcal{P}$ (i.e) $\nabla \log(P) = \frac{\nabla p}{p}$ which mean we are dividing the vector by the scalar along all dimensions
3. More numerically stable because when ∇P is very small, there are chances that 'p' is also very small. So according to the point above if we take the ratio by dividing, it is not as small.

8.4 Score Matching

Goal : Estimate the score of $s_\theta(x)$

$$\mathcal{L}_{SM} = \mathbb{E}_x \left[\left\| \theta(x) - \underbrace{\nabla(x) \log \mathcal{P}_{data}(x)}_{\text{We don't have access to it}} \right\|_2^2 \right]$$

1. Implicit Score Matching
2. Sliced Score Matching - A scalable approach to score and density estimation, Song et al, 2019

These two approaches are not commonly used now.

8.5 Score of a Gaussian Distribution (example in 1D)

For $x \sim \mathcal{N}(\mu, \sigma^2)$

So the probability distribution function is $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp - \left(\frac{(x-\mu)^2}{2\sigma^2} \right)$

Therefore, score is $s(x) = - \left(\frac{x-\mu}{\sigma^2} \right)$

Now we know that finding the score of a Gaussian distribution is easy. So we take one point and add noise to it and scale it by σ

$$\tilde{x} = x + \sigma\epsilon, \epsilon \sim \mathcal{N}(\mu, \mathcal{I})$$

Perturbation kernel

Not much is explained by the instructor. This is a facility to add noise.

$$q_\sigma(\tilde{x}) = \int \underbrace{q_\sigma(\tilde{x} | x) \mathcal{P}_{data}(x)}_{\text{Joint probability } q(\tilde{x}|x)} dx$$

This is an expression that is also considered a Joint probability because if we integrate over $\tilde{x}$ and x , we get 1.

Noise is added to every data point in the data and more the noise more the perturbations.

Estimating the score of the noised distribution

$$\mathcal{L}_{SM}(q_\sigma) = \mathbb{E}_{\tilde{x}} [\|\theta(\tilde{x}) - \nabla(\tilde{x}) \log q_\sigma \tilde{x}\|_2^2]$$

The instructor mentions a research paper that proves that the loss above is equivalent to the following loss equation.

$$\mathcal{L}_{DSM}(q_\sigma) = \mathbb{E}_{x, \tilde{x}} \left[\left\| \overbrace{s_\theta(\tilde{x})}^{\text{True score}} - \underbrace{\nabla(\tilde{x}) \log q_\sigma(\tilde{x} | x)}_{\substack{-\frac{\tilde{x}-x}{\sigma^2} \\ \text{Tractable}}} \right\|_2^2 \right]$$

8.6 Denoising Score Matching

$$\mathcal{L}_{DSM}(q_\sigma) = \mathbb{E}_{x, \tilde{x}} \left[\left\| \overbrace{s_\theta(\tilde{x})}^{\text{True score}} - \underbrace{\nabla(\tilde{x}) \log q_\sigma(\tilde{x} | x)}_{\substack{-\frac{\tilde{x}-x}{\sigma^2} \\ \text{Tractable}}} \right\|_2^2 \right]$$

However, $\mathcal{L}_{DSM} = \mathcal{L}_{SM} \underbrace{(q_\sigma)}_{\substack{\text{This is not} \\ \mathcal{P}_{data}}}$

8.7 Limitations of vanilla Denoising Score Matching

It appears that if q_σ with $\sigma \ll 1$ then q_σ is close to $\mathcal{P}_{data}$ but estimations are poor in regions where $\tilde{x}$ is not likely to appear. **What does this mean ?a**

This may be the subject of one of Lilian Wong's blog posts.

8.8 Score Matching with varying noise via NCSN - Noise Conditional Sampling Network

Score Modelling by estimating gradients of the Data Distribution, Song et al, 2019

We sample from a distribution is is easy to sample from like the Gaussian. After that we predict the score from distributions that progressively less noisy until we reach very close to $\mathcal{P}_{data}$ which is a noisy data distribution which very low σ

This is **Annealed Langevin Dynamics**

$q_{\sigma_{T-1}}$ with $\sigma_T > \sigma_{T-1}$

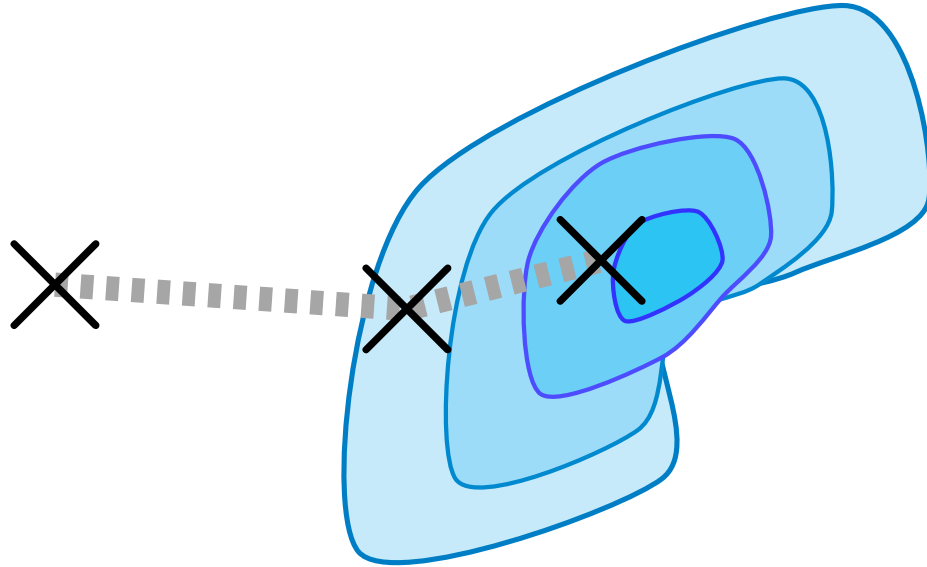


Figure 8.2:

8.9 This is Annealed Langevin Dynamics

1. Sample : $x_{\mathcal{T}} \sim \mathcal{N}(0, \sigma_{\mathcal{T}}^2)$
2. For each $\sigma_{\mathcal{T}}$ perform iterative update for K steps
3. Obtain final image x_0

$$x \leftarrow x + \frac{\alpha^i}{2} \underbrace{s^\theta}_{\text{Predicted Score to move towards } (x, \alpha^i)} + \underbrace{\sqrt{\alpha^i} \epsilon}_{\text{Sampled from a Normal Distribution}}$$

8.10 Parallel between noise (DDPM) and score (NCSN)

$$\text{DDPM} = \underbrace{\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon}_{q(x_t | x_0) = \sqrt{\bar{\alpha}_t}x_0, \sqrt{1 - \alpha_t}\text{I}}$$

$$x_t \log q(x_t|x_0) = - \frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$$

The last formula shown above links the score predicted to move from noisy to a clean image and the noise level added. They are equal.

8.11 Wiener Process

Idea : Continuous equivalent of adding increments of Gaussian noise

Wiener Process : Stochastic process with the following properties.

1. $W_0 = 0$
2. $W_T - W_s \sim \mathcal{N}(0, (t - s)\text{I})$
3. Only Increments are independent. Not the value of W_t

Now we can derive the continuous formulation of DDPM and NCSN. Here we show only the derivation of DDPM.

$$x_t = \sqrt{1 - \underbrace{\beta_T}_{\text{Noise schedule}}} x_{t-1} + \beta_t \epsilon \quad \underbrace{x_t = x_{t-1}}_{\text{Differential evolution if x as a function of everything else}}$$

$x_t - x_{t-1}$ is very small. So it is considered continuous. $x_t - x_{t-1} = [\sqrt{1 - \beta_t} - 1] x_{t-1} + \sqrt{\beta_t} \epsilon$ Why is 1 subtracted ?

$\beta_t = \beta(t)dt$ is rate at time t $x_t - x_{t-1} = [\sqrt{1 - \beta(t)dt} - 1] x_{t-1} + \sqrt{\beta(t)dt} \epsilon$
Why is 1 subtracted ?

When dt tends to 0 we have taylor expansion which is $\frac{1}{2}$ of that quantity.

$$dw = \sqrt{dt} \epsilon \text{ according to Wiener process}$$

Differential evolution

$$\widehat{dx} \sim [1 - \frac{1}{2}\beta(t)dt - 1] x + \sqrt{\beta(t)}dw$$

$dx = -\frac{1}{2}\beta(x)dt + \sqrt{\beta(t)}dw$ In other words the noising forward process brings x closer to 0 but also adds noise so that the variance converges to 1. DDPM is exactly the same process. We take a clean image and add noise in increments and after time steps 't', it converges to a normal distribution with mean 0 and variance 1.

Table 8.1: SDE Variants

Variance Preserving

Example DDPM

$$f(x, t) =$$

$$-\frac{1}{2}\beta(t)x \text{ Drift}$$

$$g(t) =$$

$$\sqrt{\beta(t)} \text{ Diffusion}$$

8.12 Training Derivation

Objective : Estimating the score $s_\theta(x, t)$

$$\mathcal{L}_{DSM} = \mathbb{E}_{t, x_0, x_t} [\Lambda_t \|\theta(x_t, t) - \nabla(x_t) \log p(x_t | x_0)\|_2^2]$$

Introducing the reverse formula

$$dx = \underbrace{\{(\beta, \gamma)dt\}}_{\text{Drift}} + \underbrace{g(t)dW}_{\text{Stochastic Diffusion}}$$

Intuition behind the reverse formula

Reverse : Transform noise back into data The paper is 'Reverse time diffusion equation models', Anderson, 1982 The VIP cheatsheets, CS 230, Deep Learnings, Amidi, 2019

The following is the result based on SDE research and this is the reason for moving from discrete to the continuous world.

$$dx = \left[(f(x, t) - \underbrace{gt^2 \nabla \log(\mathcal{P}_t(x))}_{\text{Compensation for drift}}) \right] dt + g(t) d\bar{W}$$

The SDE research uses the score in the formula to reverse. This means that the original diffusion(Brownian motion ?) added during the forward process is offset when we reverse.

CHAPTER 9

Linear Dynamical Systems

9.1 Canonical Model for Time series Data

$$x_{t+1} = \underbrace{A}_{\text{Transition Matrix}} x_t + \underbrace{B}_{\text{Control Matrix}} u_t + \underbrace{w_t}_{\text{Process noise}}$$

$$y_t = \underbrace{C}_{\text{Sensing Matrix}} x_t + \underbrace{B}_{\text{Feedthrough Matrix}} u_t + \underbrace{z_t}_{\text{Observation noise}}$$

We can think of this noise as added Gaussian noise. While linear dynamical systems have many applications they are particularly suitable for Robotics, Navigations and learning.

Example from rocket trajectory

$$x_t = \begin{bmatrix} \text{Positions} \\ \text{Velocity} \\ \text{Acceleration} \end{bmatrix} \quad x_t = [\text{Laws of motion}]$$

Example from epidemiology

$$x_t = \begin{bmatrix} \text{Susceptible} \\ \text{Exposed} \\ \text{Infected} \\ \text{Recovered} \end{bmatrix} x_t = [\text{State machine}]$$

Other applications are Randomized Control Trials, Speech Processing, Econometrics and Finance. In the case of Randomized control trials, it can also solve the $n = 1$ problem as it pertains to pain medications that a particular patient needs.

$$\underbrace{x_{t+1}}_{\text{Hidden State}} = Ax_t + B \underbrace{u_t}_{\text{Control}} + w_t$$

$$\underbrace{y_t}_{\text{Observation}} = Cx_t + Bu_t + z_t$$

+ When the parameters are known, prediction and inference are easy (like Kalman Filtering).

? How do you learn its parameters?

9.2 Main problem (informal)

Given one long trajectory

Input/Controls : $u_1, u_2, \dots, u_T$ Outputs : $y_1, y_2, \dots, y_T$

Definition: We consider two Linear Dynamical Systems to be equivalent if for any sequence of adaptively chosen inputs $u_{t+1} = f(y_1, y_2, \dots, y_T, u_1, u_2, \dots, u_T)$, **they generate the same distribution on the output up to a transformation of the noise.**

So even an adaptive adversary cannot differentiate one LDS from the other in this case because the two LDS's differ only in the hidden state which is not exposed.

Proposition: Two Linear Dynamical Systems with Gaussian noise are equivalent $\iff \exists U$ such that

$$A = U^{-1}\hat{A}U, B = U^{-1}\hat{B}, C = \hat{C}U, D = \hat{D}$$

This defines a natural parameter distance.

9.3 Main problem (informal)

Given one long trajectory

Input/Controls : $u_1, u_2, \dots, u_T$ **Outputs :** $y_1, y_2, \dots, y_T$

Can we find $\hat{A}, \hat{B}, \hat{C}, \hat{D}$ such that

$$\|A - U^{-1}\hat{A}U\|_F, \|B - U^{-1}\hat{B}\|_F, \|C - \hat{C}U\|_F, \|D - \hat{D}\|_F$$

Is there a polynomial time learning algorithm for learning ?

The most important matrix in a Linear Dynamical System is the Transition matrix. This matrix describes how the system evolves.

Widespread assumption on **Spectral Radius** often unreasonable

9.4 Prior work

Strict stability $p(A) < 1$ fails even in simple cases	Marginal stability $p(A) \leq 1$ Otherwise system would explode
No long-range correlations	

The explanation from Ankur Moitra, who is the instructor, is that discretization of ODE's create a upper-triangular matrix with ones in the diagonal. The eigen values are equal to 1.

Bounds depend on $\frac{1}{(1-P(A))}$ degrades as $P(A) \rightarrow 1$

Do long-range correlations actually restrict learning ?

9.5 observability and controllability

The observability matrix of order s is

$$\mathbf{O}_s = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \vdots \\ \mathbf{CA}^{s-1} \end{bmatrix}$$

Proposition : If it doesn't have full column rank, there is some portion of the state space we miss even over s steps.

We don't learn properly.

Proof : If we move the state x_t in some direction z then

$$\mathbf{CA}_{x_t} = \underbrace{\mathbf{CA}_{(x_t+z)}}_{\text{in the kernel of C}} \Rightarrow \text{no effect on } y_{t+1} \text{ etc.}$$

If you have a vector in the kernel of the observability matrix, it describes the direction that the state could move in but you can never see any of these changes in any of the s steps.

Definition : The controllability matrix of order s is

$$\mathbf{Q}_s = [\mathbf{B}, \mathbf{AB}, \dots, \mathbf{A}^{s-1}\mathbf{B}]$$

B is the matrix that describes the relationship between the control **U** inject and how it affects the hidden state.

A is the transition matrix

Proposition : If it doesn't have full row rank, there is some portion of the state space that cannot be reached by appropriate inputs.

Necessity of these assumptions goes back to Kalman in 1960

Recent results : Theorem : Bakshi Liu, Moitra, Yau There is a polynomially stable algorithm for learning any marginally stable Linear Dynamical System from one long trajectory from **quantitative observability and controllability** *

*(i.e) Conditioning number of bounds

Moreover these conditions are essentially minimal

Theorem : Bakshi Liu, Moitra, Yau If the controllability and observability matrices are ill-conditioned you can prove a lower bound on the sample complexity as long as they are ill-conditioned on every s .

It is information theoretically possible.

Comments : (Simchowitz, Boczar, Recht) also gave algorithms under marginal stability, **but unspecified dependence on system properties** *

Also renewed interest due to their relation to Recurrent Neural Networks. Analyzing how well gradient descent work on RNNs is going to be very, very tough. So they studied Linear Dynamical Systems. Before using LDSs as a prototype for reasoning about RNN, we need to understand their fundamental limits. (e.g) What are minimal assumptions about learnability ?

Introduced by Karl Pearson in 1894 .

Estimate moments of distribution from the sample
 Setup system of equations in unknown parameters
 Solve to compute estimates

Is there a recipe for non-stationary data ?

9.6 Blueprint

Definition The markov parameters upto order $2s + 1$ are

$$\mathbf{G} = \mathbf{D}, \mathbf{CB}, \mathbf{CAB}, \dots, \mathbf{CA}^{2s}\mathbf{B}$$

Estimate the Markov parameters from the sample
 Setup system of equations in unknown parameters
 Solve for the parameters using **Ho-Kalman Algorithm**

The HO-Kalman Algorithm

Step 1 - Form the Hankel Matrix

$$\begin{matrix} & \text{H1} & & & \\ & \left[\begin{array}{c|c|c|c} C & CAB & \cdots & CA^s B \\ CAB & CA^2 & \cdots & B \\ \vdots & \vdots & \ddots & \vdots \\ CA^s B & \cdots & \cdots & CA^{2s} B \end{array} \right] & & \\ & & & \text{H2} & \end{matrix} \quad (9.1)$$

Take the Markov parameters and organize them into a giant Hankel matrix.

Claim : $\mathbf{H} = \mathbf{O}_{s+1} \mathbf{Q}_{s+1}$

H represents the markov parameters and we stack them in a block Hankel matrix.

$$O = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \end{bmatrix} \quad Q = \begin{bmatrix} B \\ AB \\ A^2B \\ \vdots \end{bmatrix}$$

Step 2 - Singular Value Decomposition

Can we compute another factorization and show equivalence ?

$$\underbrace{H_1}_{\text{Hankel matrix with last column removed}} = U \Sigma V^T = \underbrace{(U \Sigma^{\frac{1}{2}})}_{\hat{O}} \underbrace{(\Sigma^{\frac{1}{2}} V^T)}_{\hat{Q}}$$

I can write its SVD.

$$= O_{s+1} Q_s$$

This is possible by removing the last column of the Hankel matrix. How ?

The following seems to be a Linear Algebra result.

Lemma : If O_{s+1} and Q_s have full column and row rank respectively then

$$O_{s+1} = \hat{O} T, Q_s = T^{-1} \hat{Q} \text{ for some matrix transformation.}$$